

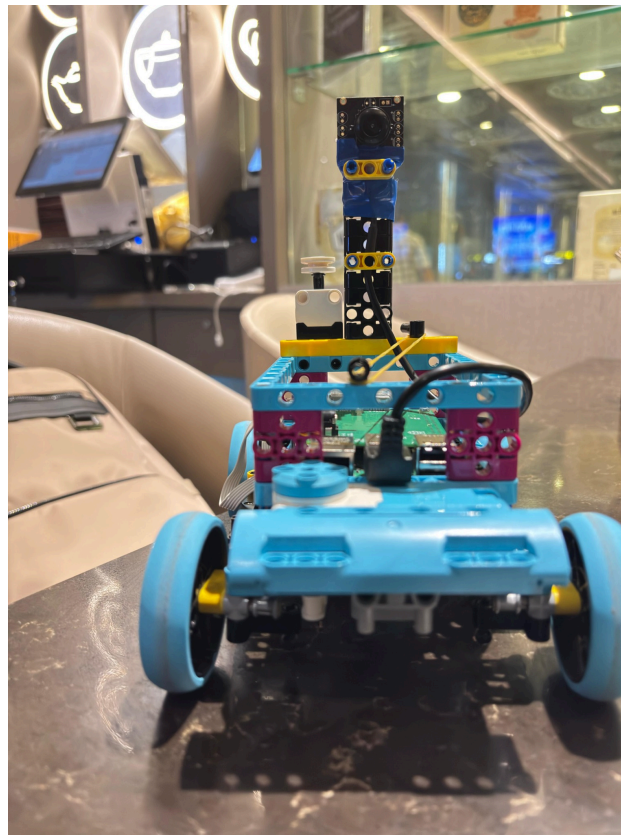


Engineering Journal

Future Engineers 2026

THE MECH MINDS

A camera-guided vehicle built with Raspberry Pi 5 and LEGO motors



Team: Ahmed Suleiman | Hajer Talib Al Salmani | Ismail Nassor

Coach Alex Savariyar

github.com/The-Mech-Minds/WRO26_Future-Engineers_TMM



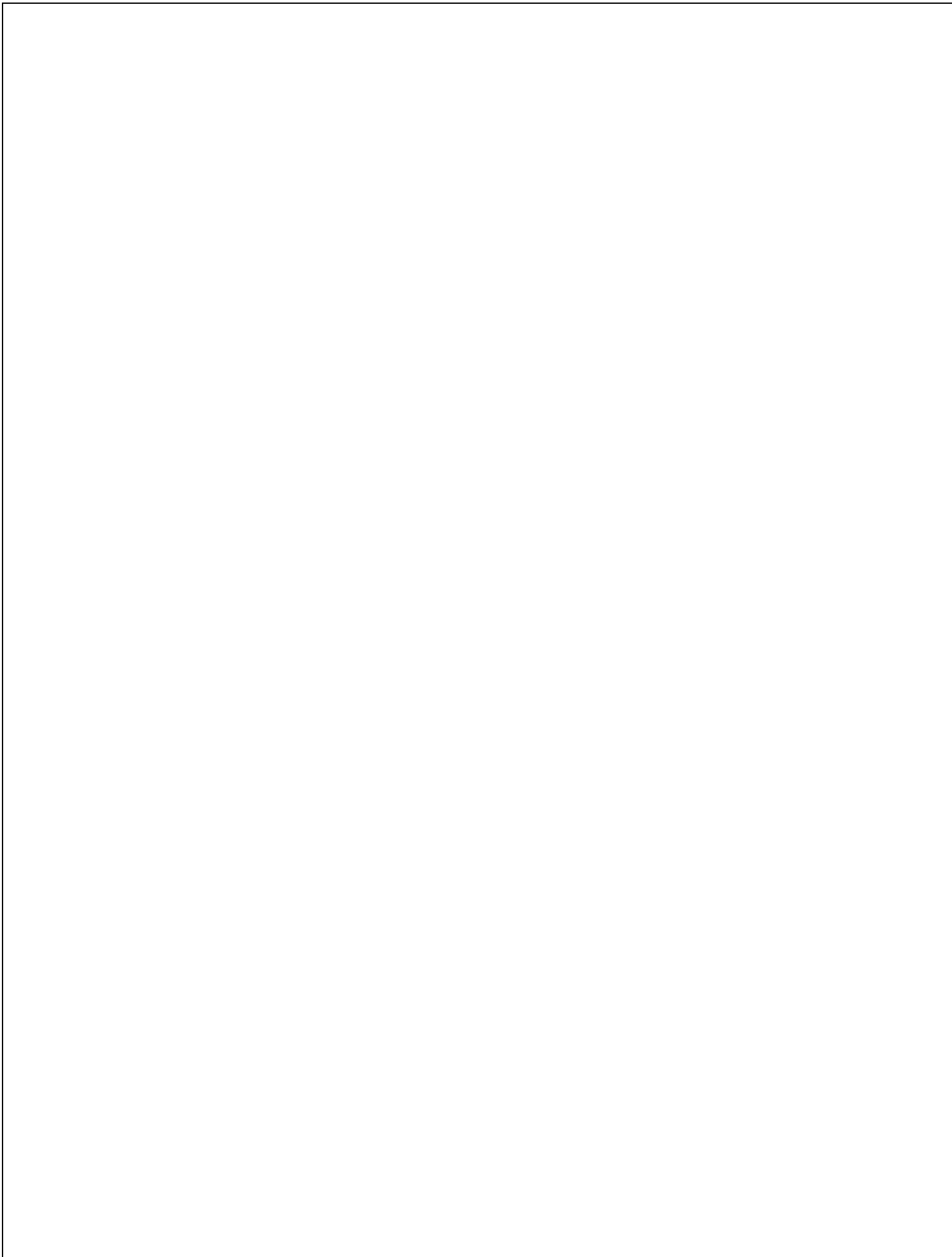
Abstract

We designed a camera-guided vehicle to navigate the WRO Future Engineers track using a Raspberry Pi 5, a Raspberry Pi Build HAT, two LEGO motors and a forward-facing Hiwonder USB camera. One motor steer the front linkage, while the other drives the vehicle. A Force Sensor starts and stops the run. The program uses OpenCV to analyse separate regions for the start lines, lap markings, coloured pillars and dark walls. It identifies clockwise or counterclockwise travel from the vertical order of orange and blue lines, then chooses a corner turn in the locked direction. A green pillar requests left steering; a red pillar requests right steering. During unobstructed travel, the program corrects away from a detected side wall. The line counter records rearmend orange or blue crossings and requests a stop at crossing 13 after three reported laps.

The vehicle followed an earlier Raspberry Pi 4B and Arduino prototype that used serial motor commands, contour-based wall centring, a blue-only lap detector and timed reverse parking. We changed the control hardware and separated camera processing, race decisions and motor commands into three Python files. This journal explains those decisions with photographs and diagrams and identifies what has not yet been demonstrated by the available evidence: verified motor calibration, confirmed power wiring, complete lap and pillar-clearance tests, an underside photograph and an autonomous driving video. The current code does not perform parking.

Keywords

Autonomous vehicle; colour segmentation; HSV; wall detection; Raspberry Pi 5; Build HAT; WRO Future Engineers.



Contents

Engineering Journal | The Mech Minds

Introduction	7
Vehicle and mechanical design	8
Power and sensor architecture	9
Software architecture and code flow	10
Image processing method	11
Direction and lap strategy	12
Wall and pillar navigation	13
Build HAT motor control	14
Design evolution and decisions	14
Testing and limitations	15
Reproduction guide	16
Technical references	18



1 Introduction

The aim of this project is to develop a small autonomous vehicle that interprets the WRO Future Engineers field through a forward-facing camera and makes steering decisions during a run. The course requires the vehicle to follow the road through corners while responding to coloured traffic pillars and floor markings. The start position does not justify assuming one travel direction, so the controller first estimates clockwise or counterclockwise motion from the camera image. Our engineering question is whether the same sensing and motor structure can support both directions while keeping the vehicle inside the boundaries. This journal documents the present design and its limitations against the task described in the official rules. [1]

1.1 What the vehicle observes

The camera supplies one image after another to the Raspberry Pi 5. Our vision program separates this image into regions chosen for specific observations. Colour masks identify orange and blue floor markings and red and green traffic pillars, while dark-pixel ratios in three other regions estimate the front, left and right walls. Each successful frame produces a direction estimate, a possible obstacle, wall measurements, the line count and a suggested side correction. The main program interprets these outputs in priority order and then calls the motor controller. Keeping image measurement separate from the action decision allows us to examine a camera overlay before changing steering parameters.

1.2 Scope of the submitted revision

The present revision uses a Raspberry Pi 5, Python and OpenCV for perception and decision making. A Hiwonder USB camera supplies video, and a LEGO Force Sensor on Build HAT port C provides the physical start and stop input. Two LEGO motors perform steering and propulsion. The software names ports A and B as the respective motor defaults, although their physical connections still need confirmation on the competition vehicle. The code reports laps at line counts 4, 8 and 12 and uses count 13 as its stopping reference. It has no autonomous reverse-parking sequence. These distinctions define what can be concluded from the source and what still requires field evidence.

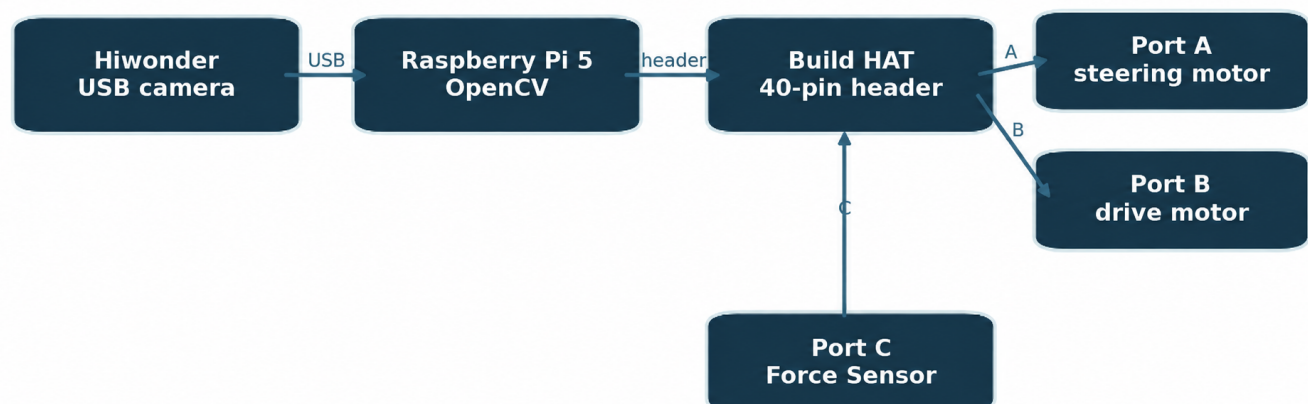


Figure 1. Current information and control connections. Motor ports A/B are configurable defaults.

This distinction matters: the repository contains valid source and construction photographs, but it does not establish a measured competition run. The evidence and missing validation are separated in section 10.

2 Vehicle and mechanical design

We assembled the current vehicle around a compact LEGO Technic frame carrying the Raspberry Pi 5, Build HAT, front steering linkage and driven wheels. The camera stands above the chassis so that its view includes floor markings, traffic pillars and the front wall area. This mounting height creates a practical compromise: a low camera would give more road detail close to the wheels, while a high camera makes distant boundaries easier to see. In the submitted design the camera remains fixed to the frame, so the image regions can be calibrated for one repeatable viewpoint. One LEGO motor rotates the steering linkage and a second motor drives the wheels. [2, 3]

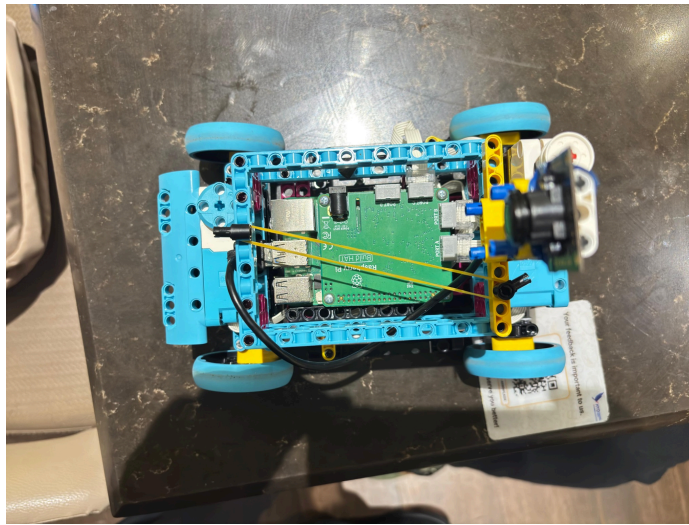


Figure 2. Current chassis viewed from above: central electronics and forward camera mount.

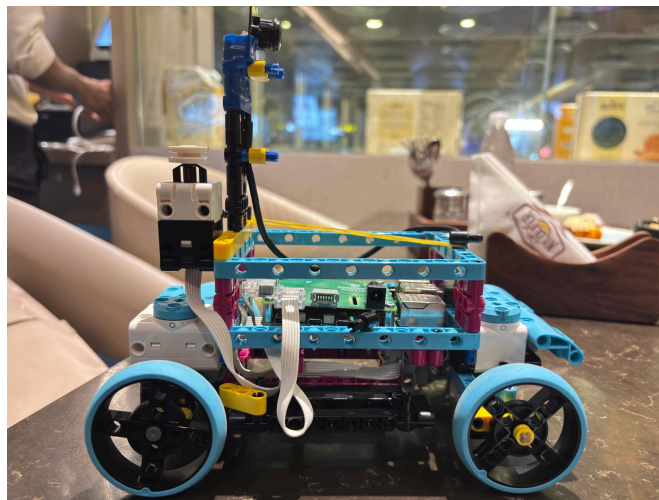


Figure 3. Side view showing wheel layout, steering assembly and camera height.

We separated steering from propulsion because the vehicle must adjust its heading without repeatedly reversing the drive motor. The motor controller commands an absolute steering-motor position, but that position is not necessarily equal to the angle of the road wheels. The linkage ratio, gear play and mechanical end stops determine the actual angle. Even with the same command, wheel slip, axle friction, frame flex and uneven mass distribution can change the turning radius. The photographs

establish how the assembly is arranged; they do not establish numerical torque, wheel-angle or turning-radius performance. Those measurements should be made with the assembled robot before fixing the final turn values.

3 Power and sensor architecture

The Hiwonder camera connects through USB to the Raspberry Pi 5, which reads and processes its video frames. The Raspberry Pi Build HAT connects to the Pi header and provides the motor and sensor interface. In the present source, `main.py` creates a Force Sensor on port C, while `vehicle.py` gives ports A and B as defaults for steering and drive. These port names are useful for understanding and reproducing the program, but the cables and the installed power supply must be checked on the physical competition robot. A mismatch between a configured port and a connected motor could make an otherwise correct vision decision produce the wrong action. [3–5]

Table 1. Current hardware connections and their verification status.

Connection	Purpose	Status
Camera → USB	Video input	Visible in photographs; camera index 0 in code
Pi header → Build HAT	Motor and sensor controller	Board present in current chassis
Port A → steering	Target motor position	Provisional code default
Port B → drive	Forward motor speed	Provisional code default
Port C → Force Sensor	Start / stop	Explicit in <code>main.py</code>

3.1 Power verification

Reliable motor operation depends on the power path as much as on the program. Raspberry Pi documentation specifies an external +8 V centre-positive supply capable of 48 W for the Build HAT motor arrangement and explains that the HAT can supply the Pi when connected correctly. We use that specification as a reference; the present photographs do not confirm the battery, connector polarity or measured voltage of our assembled vehicle. Before a competition run we need to record the installed supply and inspect voltage under steering and drive load, since a voltage drop can interrupt processing or reduce motor response. The operating-system image also needs verification because the manufacturer notes that Build HAT support for Raspberry Pi OS Trixie is pending. [3]

3.2 Placement and repeatability

The camera position is part of the sensor calibration. A small change in tilt or height moves a floor marking into a different part of the frame and can cause a fixed detection region to miss it. We therefore define the image regions relative to a 640×480 frame and inspect them with the live debug overlay whenever the camera mount is reassembled. The Force Sensor lets the operator hold the robot stationary while the camera feed becomes available, then start the run with a deliberate press. Its second press is checked as a stop request during navigation.

4 Software architecture and code flow

We divided the current program into vision.py, main.py and vehicle.py to give sensing, decision making and actuation clear responsibilities. The vision module reads the camera and returns interpreted measurements; the main module chooses the action to take from those measurements; the vehicle module translates that action into Build HAT motor commands. This separation helps us adjust a colour threshold without rewriting the steering functions and lets us inspect the detection overlay before a motor test. The module interfaces use the Build HAT motor and sensor classes described in the Raspberry Pi references. [3–5]

Table 2. Functions and outputs of the three current Python modules.

Module	What it produces
vision.py	Frame, direction, pillar, walls, line event, lap state, debug overlay
main.py	Priority decisions, corner loop, button and Q handling, cleanup
vehicle.py	Build HAT steering position, drive speed, stop and centre

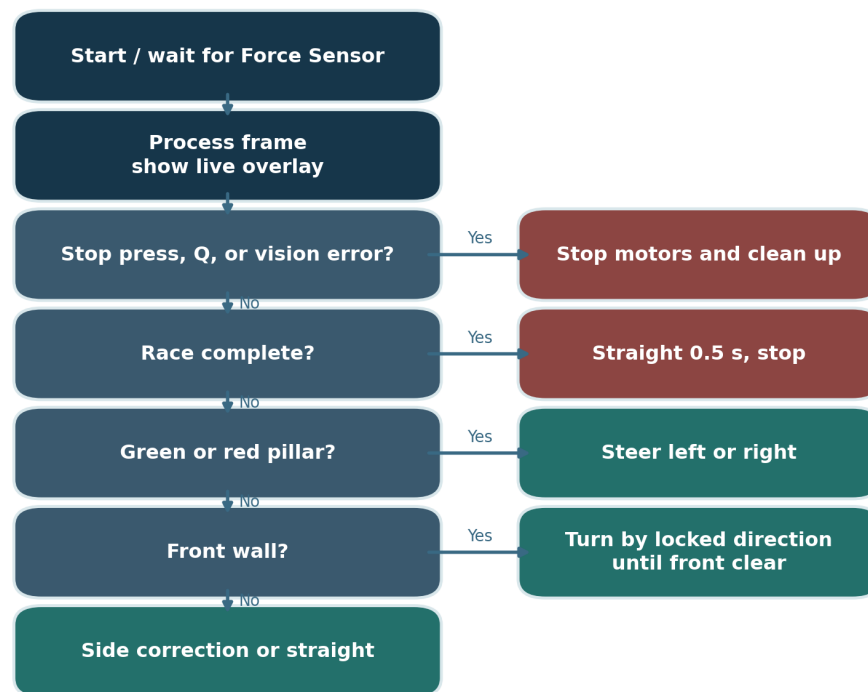


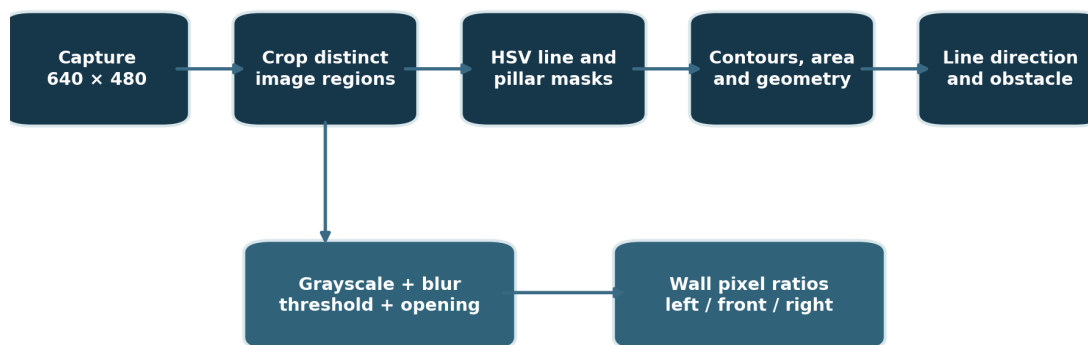
Figure 4. Main program decision flow. Red and green pillar checks precede front-wall turning.

At startup, the drive motor stops and the camera feed remain visible while the program waits for the first button press and its release. Each navigation cycle then checks the second press stop condition, requests a new vision result and displays a debug frame. When the vision module reports race completion, the program briefly drives straight and exits. Otherwise, the decision order is green pillar, red pillar, front wall and normal side correction. This order matters because a visible pillar can demand steering even when a front wall is also present. For a front wall, the chosen turn continues through new camera frames until the wall is classified as clear, with button and camera checks inside the turn loop. The current code

has no turn timeout, and when direction remains UNKNOWN the front-wall case defaults to left. These conditions require explicit testing or a safer fallback before depending on the program at full speed.

5 Image processing method

The vision process opens the USB camera with OpenCV VideoCapture, requests 640×480 pixels at 30 frames per second and resizes each successfully read frame. The resized image is divided into regions assigned to direction selection, line counting, pillar detection and left, right and front walls. This is a practical way to limit the number of irrelevant pixels considered by each detector. A coloured object outside its region is ignored even if its colour mask is correct, so the regions depend on the mounting position, field view and expected approach distance. Their outlines are shown in Figure 6 and should be checked on the live camera image after a mechanical change. [6]



Colour branch and wall branch are combined into one frame result for main.py.

Figure 5. Two image-processing branches contribute to the same per-frame result.

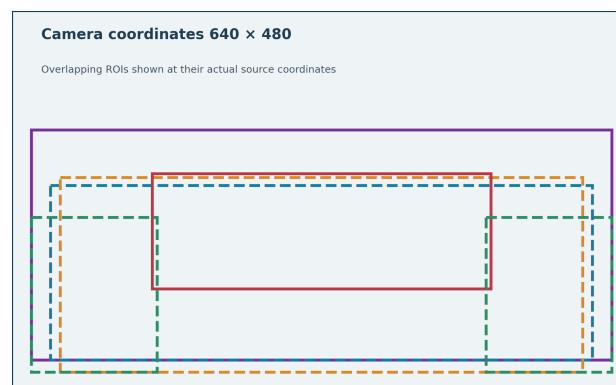


Figure 6. Source regions at 640×480 : purple direction, blue lap, orange pillar, green side walls and red front walls.

5.1 Why HSV and contours

We convert BGR camera pixels to HSV because hue allows the program to specify separate intervals for orange, blue, green and red. Saturation and value limits reduce responses from pale floor patches and poorly illuminated pixels. The `inRange` operation turns pixels inside an interval into a binary mask; morphological opening removes isolated detections, while closing can join small gaps in a pillar mask. The program then finds external contours and evaluates area, bounding dimensions and position. These

standard OpenCV operations explain how image pixels become candidate lines or pillars, but the numerical ranges and region coordinates are our implementation choices and need verification under the event lighting. [7–10]

6 Direction and lap strategy

The navigation direction initially has the value UNKNOWN. To resolve it, the program searches a relatively large image region for orange and blue line contours, each with at least 80 px² area. It discards a blue contour that looks tall and narrow instead of line shaped. If both colours are present and their vertical centres differ by 20 pixels or more, the lower line in the image is treated as closer to the vehicle. Orange below blue proposes CLOCKWISE, whereas blue below orange proposes COUNTERCLOCKWISE. The same proposed direction must appear on three successive frames before it is locked. This repeated observation is intended to reduce a false lock from one noisy frame; its effectiveness on both field starts still needs to be demonstrated. [7–10]

Table 3. Camera observations used to determine and lock the travel direction.

Observation	Program response
Only one line or separation < 20 px	Reset consecutive confirmation
Orange centre lower in image	Clockwise candidate
Blue centre lower in image	Counterclockwise candidate
Three identical observations	Lock direction for the run

6.1 Debounced line events

Lap counting uses a smaller image region. After direction is locked, the starting line must disappear before counting is enabled. A valid orange or blue marking then produces one event; the counter remains locked for as long as the marking is visible. Only five consecutive frames without that marking rearm the detector. In this way, a broad line occupying several consecutive frames should cause one crossing event. The current code reports completed laps after counts 4, 8 and 12. At count 13 it raises `race_complete`, after which `main.py` commands straight motion for 0.5 seconds and stops. The chosen number of crossings and the final travel interval need to be checked against continuous footage of an actual run.

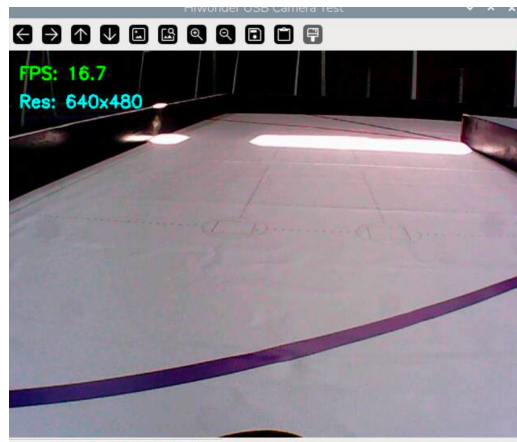


Figure 7. Development camera frame showing a blue line. Its overlay is from an earlier test program.

The 13th crossing is a stop reference in the current implementation. There is no reverse parking state, so the report does not claim parking performance.

7 Wall and pillar navigation

The wall detector converts each camera frame to grayscale, smooths it with a 5×5 Gaussian blur and applies an inverted binary threshold of 100. Morphological opening removes small isolated dark patches. In each wall region, the proportion of foreground pixels is calculated by dividing the nonzero mask count by region area. A front ratio above 0.15 produces FRONT_WALL, while a left or right ratio above 0.22 marks that side as close. Ratios make the measurement independent of region pixel count, but they do not directly measure physical wall distance. Changing illumination or camera angle can therefore change the decision without any change in actual clearance. [8, 9]

Table 4. Wall and traffic pillar observations and their navigation responses.

Per-frame result	Navigation response
Front ratio > 0.15	FRONT_WALL: turn right for CLOCKWISE, otherwise left until clear
Only left ratio > 0.22	RIGHT_CORRECT
Only right ratio > 0.22	LEFT_CORRECT
Both side ratios high or both low	STRAIGHT
GREEN pillar	Left steering takes priority over front wall
RED pillar	Right steering takes priority over front wall

Traffic pillars are detected separately in the obstacle region by HSV colour masks. Red requires two hue intervals because its hue values wrap around the scale. A 3×3 morphological closing operation joins small breaks in the segmented object before contour evaluation. The current candidate rules require at least 1000 px² for a green pillar or 1800 px² for a red pillar, a height of at least 35 px and a width no more than 1.8 times the height. If valid red and green candidates appear together, the larger contour determines the reported obstacle. Green causes a left steering command and red causes a right steering command in main.py; both precede the general front-wall action. [7–10]

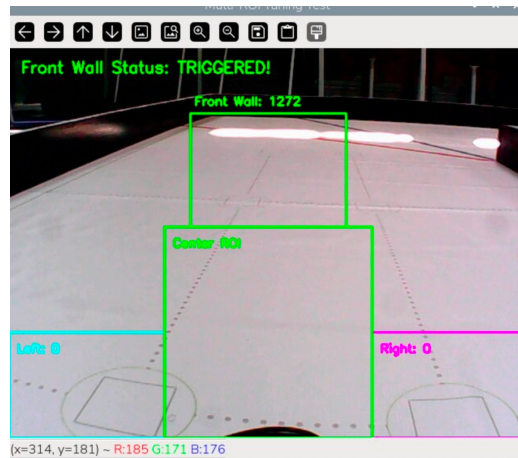


Figure 8. Earlier test overlay indicating a front wall; threshold numbers shown belong to that test.

A correct colour label is only the first part of safe obstacle passing. The current code issues a steering direction while the pillar is visible, but it does not hold an avoidance state after the pillar leaves the camera region. It also has no measured minimum clearance from the side wall during the bypass. We need continuous trials with both pillar colours, including pillars placed near corners, to determine whether the steering persists long enough and whether the vehicle returns to the road safely. Figure 8 is evidence of an earlier wall overlay, not a record of a complete current obstacle run.

8 Build HAT motor control

The Vehicle class creates separate Build HAT Motor objects for front steering and drive. Left, right, smaller correction and straight actions are represented as steering positions relative to a configured centre. When the desired position differs from the previous target, the class sends `run_to_position` with `blocking=False`. This avoids waiting for steering movement before the next camera frame. The drive motor is started at its configured speed and is not restarted on every iteration. At exit, `stop()` stops both motors and `centre()` request the central steering position. These commands describe software intent; the actual response must be checked at the wheels. [3, 4]

Table 5. Build HAT motor settings and the physical calibration required.

Parameter	Published default	What to calibrate
Steering port / drive port	A / B	Check actual connectors
Centre / full turn / correction	0° / $\pm 25^\circ$ / $\pm 10^\circ$	Measure wheel centring, end stops and effective turn
Steering speed / drive speed	30 / 20	Verify steering response and forward direction
Steering direction	+1	Set -1 if left and right are reversed

8.1 Safe calibration sequence

Calibration should begin with the drive wheels lifted and the front linkage physically centred. After confirming which motor occupies each Build HAT port, we should command small left and right movements and inspect the actual wheel directions and mechanical end stops. Drive rotation should then be checked at a low speed; a negative configured speed may be necessary if the physical forward direction is reversed. Only after these checks should a supervised low-speed track trial be used to change the correction angle and the full-turn angle one parameter at a time. This sequence describes work to perform, not measurements already obtained. The source validates target and speed ranges before creating the motors, but nonblocking commands cannot by themselves confirm that the mechanism reached a requested position. [4]

9 Design evolution and decisions

Our earlier vehicle combined a Raspberry Pi 4B with an Arduino, a hobby steering servo and a DC motor arrangement. The Pi made visual decisions and transmitted servo and motor values over a 9600-baud UART link. The earlier camera logic tracked blue line crossings with a 1.5-second cooldown, used the two visible wall contours to calculate a proportional centring error and retained a turn command for 0.4 seconds when both walls temporarily disappeared. After its twelfth blue crossing it entered a timed

forward and reverse parking sequence. This earlier implementation is preserved as `src/autonomous_main.py` and provides a record of what we changed, rather than an alternative description of the current robot.

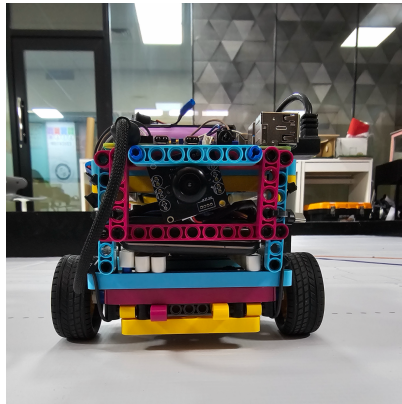


Figure 9. Earlier Pi 4B and Arduino vehicle, preserved in the project archive.

Table 6. Design changes from the earlier Pi 4B and Arduino prototype.

Reason for revision	Result in current code
Fixed right turn assumed clockwise travel	Camera-based direction lock selects both clockwise and counterclockwise corners.
Direct servo tests had jitter; separate Arduino actuation added wiring and serial commands	Pi 5 + Build HAT controls LEGO steering and drive motors directly.
Blue-only cooldown could limit handling of two line colours	Orange/blue contours and clear-frame rearm handle both colours.
Timed reverse parking belonged to the earlier drivetrain	Current code ends at crossing 13; parking must be implemented and tested separately.
Short turn memory helped loss of wall visibility	Not carried over; corner timeout and wall-loss fallback are open work.

The current revision moves motor actuation to the Pi 5 and Build HAT with LEGO motors, removing the serial command boundary and the separate Arduino controller. We also added camera-based direction selection so one program can choose the appropriate corner direction for either start, and revised line counting to observe orange as well as blue with a clear-frame rearm condition. These decisions address specific limits of the older code, but they introduce different risks. Direction can remain UNKNOWN, a front-wall turn has no timeout, a pillar bypass is not held after visual loss, and the former reverse-parking routine has not been carried over. We therefore describe this as a change in architecture and strategy, not as proof of improved field performance.

10 Testing and limitations

The supplied photographs document the visible construction of the present LEGO vehicle and its camera mount. The available camera captures show the field of view, a blue line and overlays from an earlier front-wall test. They are useful evidence for mechanical arrangement and the development of image detection, but they are not continuous evidence of the present program completing a run. To assess the strategy fairly, we separate what the repository implements from what we have verified with actual track

observations. The WRO documentation criteria also require a reproducible account of the vehicle and its operation. [1]

Table 7. Available evidence and further tests required for the current vehicle.

Test	What exists	Additional evidence needed
Both directions	Code and ROI figure	Two start-layout videos with locked direction logs
Corners and wall response	Development overlay stills	Continuous corner video and front/side ratios
Red / green pillar	HSV detection code	Video of both colours; measured clearance
Three laps and stop	Counter state in code	Continuous run and crossing log through 13
Motor and power	Robot photos and program defaults	Port photo, power rating, motor-angle calibration
Emergency exits	Button / Q branches	Observed stop and disconnect trials

10.1 Failure points to address

The first failure point concerns direction: if a front wall arrives before the program has locked clockwise or counterclockwise, the current main loop chooses left. A second issue is the lack of a timeout inside the corner turn, which can keep steering if FRONT_CLEAR never appears. Pillar avoidance stops when the relevant colour is no longer detected, even though the vehicle may not yet have passed the object. The initial button-wait loops also precede the main try/finally cleanup block, and the final 0.5-second straight movement may be too long or too short for a particular track. We need measured trials to select safe responses rather than assuming the code alone resolves these conditions.

The next evidence set should include two start layouts with the direction displayed, continuous corner and pillar run, a logged sequence through the thirteenth crossing, motor-port and power photographs, and observed emergency-stop trials. A current underside view, a team photograph and autonomous driving video would also complete the broader documentation package described by the WRO rules. None of these missing items should be represented as already tested. [1]

11 Reproduction guide

The project repository contains the three current Python modules, the README, photographs of the vehicle and camera development images. Earlier Pi 4B and Arduino material is kept under archive/pi4-arduino, which distinguishes its UART and power arrangement from the present vehicle. This separation matters when another team or judge tries to connect a circuit diagram to the program actually used on the robot. The file map below identifies where to inspect the image thresholds, the decision priority and the motor settings.

Table 8. Repository files needed to inspect and reproduce the vehicle.

File or folder	What another team can inspect
src/vision.py	Camera settings, image regions, masks, thresholds and line state

src/main.py	Start/stop, action priority, corners and race end
src/vehicle.py	Motor port and position defaults; environment overrides
v-photos/	Five exterior views of the current vehicle
docs/images/	ROI diagrams and development camera captures
archive/pi4-arduino/	Previous prototype design images

11.1 Build and run sequence

A reproduction should begin with the photographed LEGO frame, steering linkage and camera mount. The USB camera and Build HAT can then be connected, followed by confirmation of sensor port C and the actual steering and drive motor ports. The installed motor power source and operating-system support need to be verified before applying drive power. After installing the program dependencies, the camera overlay should be inspected before the wheels are placed on the track. With the drive wheels lifted, the operator can verify the steering centre, both turn directions and forward motor rotation. The complete main program should then be tried under supervision at low speed, recording the frame overlay and observed vehicle movement together.

The README remains the source for current numeric settings, while this journal explains why those settings exist and which tests are still needed. When a threshold, motor port or steering limit changes, the report should be revised alongside the code. A reproducible engineering record depends on that agreement between the photographed hardware, the documented algorithm and the observed run. [1]

12 Technical references

These references explain the methods and interfaces used in the source. The numeric thresholds and engineering decisions in this report are the team implementation, not values prescribed by these references.

[1] World Robot Olympiad Association. WRO Future Engineers Self Driving Cars Game Rules 2026, section 7 and Appendix C. <https://wro-association.org/wp-content/uploads/WRO-2026-Future-Engineers-Self-Driving-Cars-General-Rules.pdf>

[2] Raspberry Pi. Raspberry Pi 5 Product Brief. <https://datasheets.raspberrypi.com/rpi5/raspberry-pi-5-product-brief.pdf>

[3] Raspberry Pi. Build HAT documentation, connections, power and Python setup.
<https://www.raspberrypi.com/documentation/accessories/build-hat.html>

[4] Raspberry Pi Build HAT Python Library. Motor class reference: run_to_position, start, stop.
<https://buildhat.readthedocs.io/en/latest/buildhat/motor.html>

[5] Raspberry Pi Build HAT Python Library. ForceSensor class reference.
<https://buildhat.readthedocs.io/en/latest/buildhat/forcesensor.html>

[6] OpenCV. Getting Started with Videos: VideoCapture and frame reads.
https://docs.opencv.org/4.13.0/dd/d43/tutorial_py_video_display.html

[7] OpenCV. Changing Colorspaces: BGR to HSV.
https://docs.opencv.org/4.10.0/df/d9d/tutorial_py_colorspaces.html

[8] OpenCV. Thresholding Operations using inRange.
https://docs.opencv.org/4.x/da/d97/tutorial_threshold_inRange.html

[9] OpenCV. Morphological Transformations: opening and closing.
https://docs.opencv.org/4.13.0/d9/d61/tutorial_py_morphological_ops.html

[10] OpenCV. Contour Features: area and bounding rectangles.
https://docs.opencv.org/4.11.0/dd/d49/tutorial_py_contour_features.html

Project source and photographs: https://github.com/The-Mech-Minds/WRO26_Future-Engineers_TMM